M04-L01 · Autenticación y seguridad

Alberto Basalo

TESTING INTELIGENTE

Going Pro

> *Una petición nueva puede cambiar las antiguas reglas.*

CONNECTIONS

- Toda la suite pasa. ¿Sabes **qué puede hacer cada usuario**?
- Una regla nueva rompe pruebas antiguas. ¿**Quién** se ha equivocado?

CONCEPTS

Identidad, sesión y permiso

- **Identidad**: reconoce al usuario.
- **Sesión**: mantiene ese estado entre peticiones.
- **Permiso**: decide quién puede hacer qué.

El rol sale **del back**, con la identidad. El cliente no lo decide.

> *Ojo! Una prueba de acceso no demuestra que la web sea segura.*

CONCEPTS

Autenticación y autorización

- **Quién** eres → autenticación. **Qué** puedes hacer → autorización.
- La interfaz front-end oculta al usuario las acciones que no puede hacer.
- Pero **no protege nada**: es el back quien lo controla.

> *Prueba las dos capas: lo que ve cada rol y lo que la API responde.*

CONCEPTS

Una regla nueva cambia las antiguas

- La spec nueva **nombra las specs afectadas**, antes de escribir código.
- Las pruebas **antiguas** fallan, pero el código cumple la regla **nueva**.

> *Una **regresión** no siempre es un error del código.*

CONCEPTS

Enmendar el pasado

Cuando el origen es una `spec` desfasada, la corrección sigue un orden:

1. **Spec**: `spec.md` y `.feature` dicen la regla nueva.
2. **Pruebas**: la preparación sigue a la spec.
3. **Aserciones**: no se debilitan ni se quitan.

> *La spec antes que la prueba. Si no, la prueba pasa y la documentación se desactualiza.*

CONCRETE PRACTICE

La petición: dos roles

> *Hoy todo usuario con sesión puede hacerlo todo. Queremos dos roles:* `operator` *y* `passenger` *.*
>
> *Un operador gestiona cohetes, lanzamientos y reservas. Un pasajero se registra solo y puede consultar cohetes, lanzamientos y plazas libres. No puede cambiar nada ni ver los datos de otros pasajeros.*
>
> *El menú muestra el nombre y el rol. La web no ofrece al pasajero las acciones que no puede hacer, y el sistema las rechaza.*
>
> *El registro crea siempre un pasajero. El back crea un operador al arrancar, con datos del entorno.*

CONCRETE PRACTICE

P1 · La spec `S0006` con `/spec-builder`

En una conversación nueva, usa `/spec-builder` con la petición. Antes de aprobar, comprueba que:

- El registro crea un `passenger`. Sin token, 401. Un `passenger` que escribe, 403.
- El `operator` nace en el back, con datos del entorno y la contraseña como hash.
- Una sección nombra las specs afectadas: `S0002` a `S0005`. No las cambia.

CONCRETE PRACTICE

P2 · Las pruebas con `/test-builder`

En una conversación nueva, usa `/test-builder` con `S0006`.

- La semilla de `global-setup.ts` entra como `operator`.
- Las pruebas del `passenger` registran un usuario único.
- No cambian las pruebas de `S0001` a `S0005`.

Pasan las pruebas de `@S0006`. La regla funciona aislada. Eso no basta.

CONCRETE PRACTICE

P3 · Toda la suite con `/bugs-hunter`

En una conversación nueva, usa `/bugs-hunter` con `S0006` y toda la suite.

- Fallan las pruebas de `S0002` a `S0005`: entran con `existingUser`, que ahora es un `passenger`.
- Unos fallos son 403 en la API. Otros son timeouts: el control no existe para el `passenger`.
- En la pausa del triage: una causa, el código descartado y dos agentes, `spec` y después `pruebas`.

CONCRETE PRACTICE

P4 · Enmendar y explicar el alcance

Aprueba el veredicto. `fix-bugs` cambia las specs y los helpers, y la suite pasa.

- Revisa el diff de `spec.md` y `.feature` de `S0002` a `S0005`.
- Comprueba que las pruebas no cambian y que ninguna aserción se debilita.
- Haz un solo commit.

Comprobado	Fuera
401 sin token y 403 del `passenger`	El arranque del `operator`: solo pruebas unitarias
Lo que ve cada rol en la web	Asignar roles y otros roles
`S0002` a `S0005` con el `operator`	Fuerza bruta y caducidad del token

CONCLUSION

Una regla nueva revisa las antiguas

El triage encontró el origen en las specs, no en el código. La corrección las enmendó antes que las pruebas.

Antes de aceptar la regla

¿Qué **specs anteriores** cambia, y **cómo lo has sabido**?

Siguiente paso: la sesión que se repite

57 fallos esperaron 30 s cada uno, y cada prueba hace su propio login. En L02 guardarás la sesión de cada rol y la reutilizarás.

Alberto Basalo